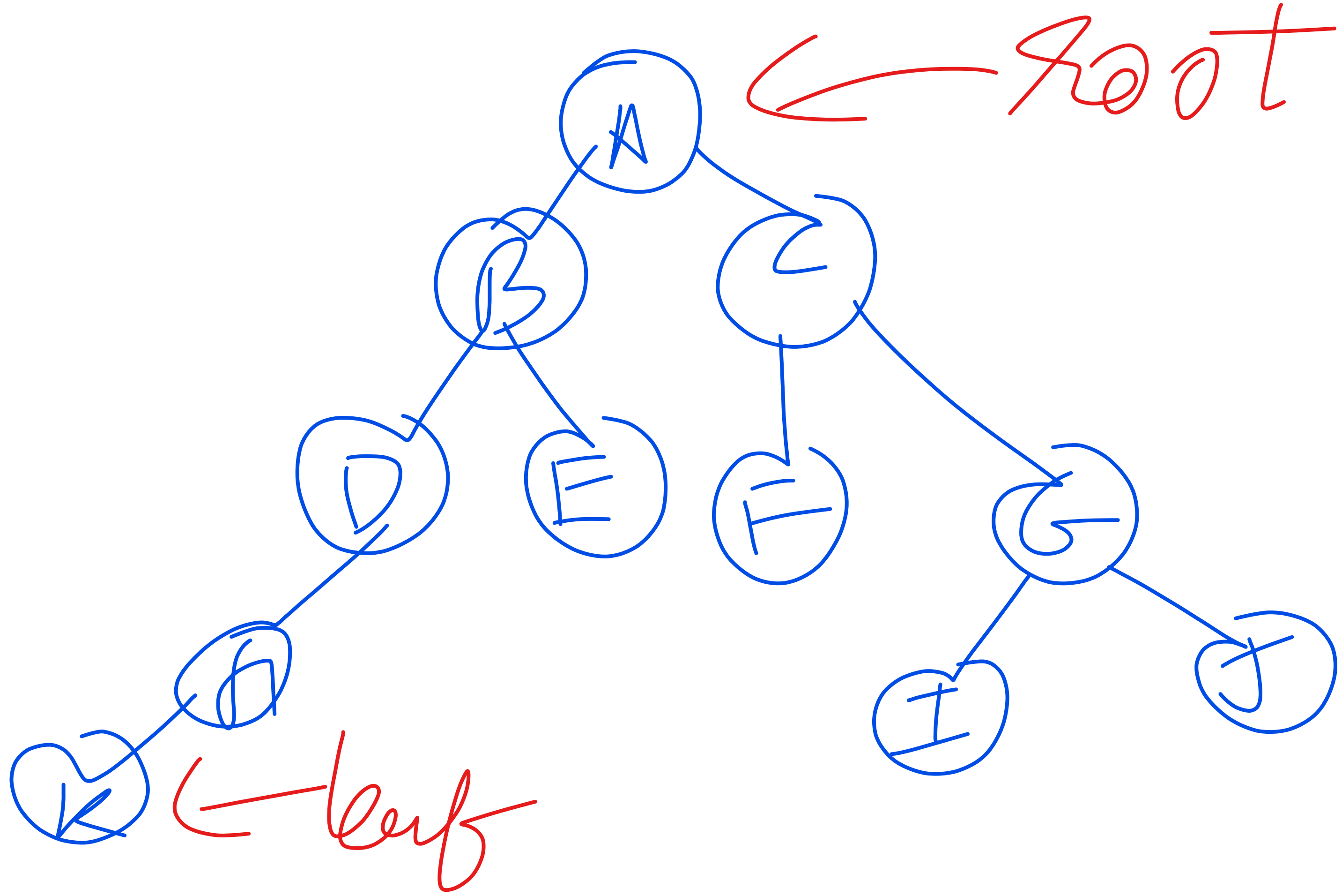
Tree

TREE

Binary Tree at most 2 child nodes

- Terms
  - Depth
  - Height
  - Subtree
  - Ancestor
  - Leaf node
  - Root node
  -



Types of Binary Tree

- Complete all leaf nodes are at same depth
- Nearly complete all leaf nodes are at same depth other than one +1 or -1
- Strict each node would either have 2 child or none
- Binary Search Tree used to store data for better search

  follows 2 rules
  1.element < parent:left
  2.element>=parent:right

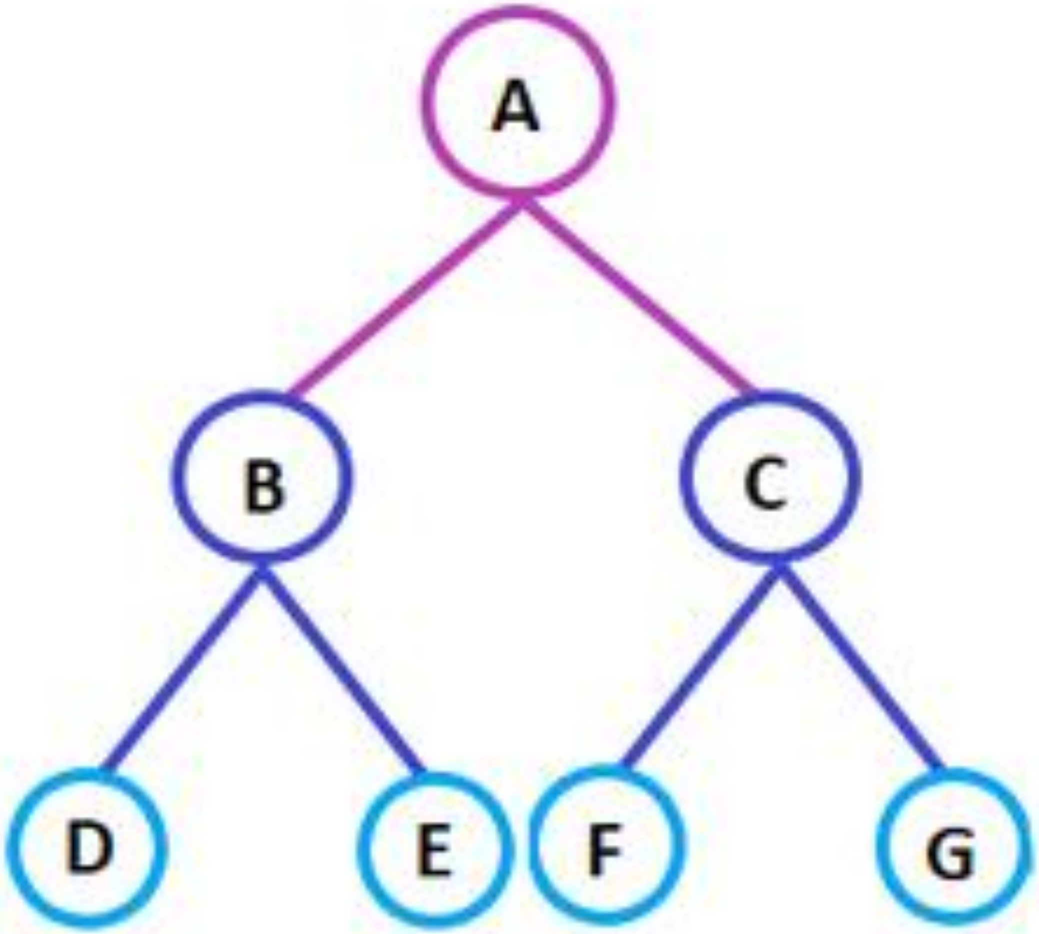
Construct Binary Search Tree

- 10,2,7,1,20,15,11,22,28

Construct Binary Search Tree

- 50,25,12,60,80,55,50,4,6,28

Tree Traversal



Construct tree

- Inorder sequence: D B E A F C
- Preorder sequence: A B D E C F

Construct tree

- preorder [3,9,20,15,7]
- inorder = [9,3,15,20,7]

Construct tree

- Inorder Traversal : { 4, 2, 1, 7, 5, 8, 3, 6 }
- Preorder Traversal: { 1, 2, 4, 3, 5, 7, 8, 6 }

Construct tree

- in[]   = {4, 8, 2, 5, 1, 6, 3, 7}
- post[] = {8, 4, 5, 2, 6, 7, 3, 1}

Construct tree

- inorder = [9,3,15,20,7]
- postorder = [9,15,7,20,3]

Construct tree

- Inorder Traversal : { 4, 2, 1, 7, 5, 8, 3, 6 }
  Postorder Traversal : { 4, 2, 7, 8, 5, 6, 3, 1 }

Threaded Tree

- Inorder traversal of a Binary tree can either be done using recursion or with the use of a auxiliary stack. The idea of threaded binary trees is to make inorder traversal faster and do it without stack and without recursion. A binary tree is made threaded by making all right child pointers that would normally be NULL point to the inorder successor of the node (if it exists).

special version of BST enhanced for inorder sequential traversal without any recursion or jump needed.
need:
traversal of data in sequential manner as most info are sequential in nature.

Threaded Tree

- Single Threaded: Where a NULL right pointers is made to point to the inorder successor (if successor exists)

Threaded Tree

- Double Threaded: Where both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively. The predecessor threads are useful for reverse inorder traversal and postorder traversal.

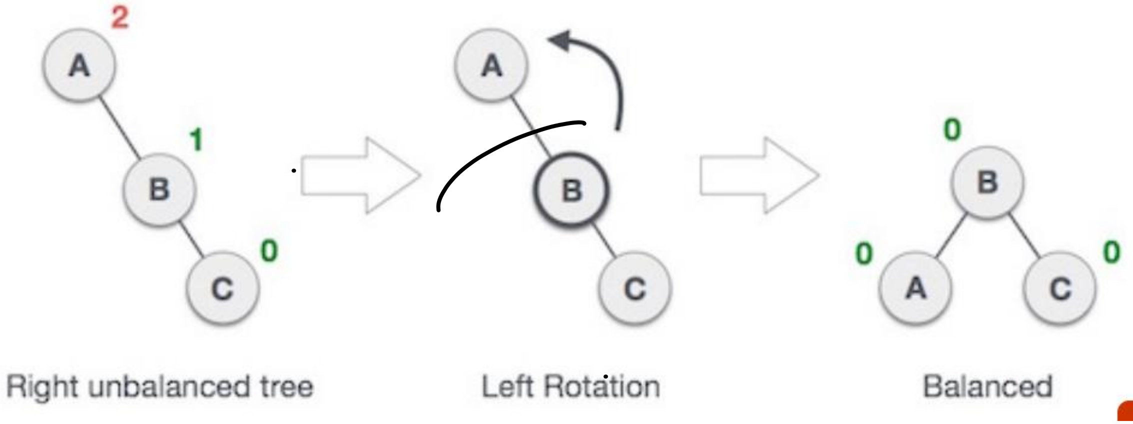
AVL TREE

self balancing tree, nearly balanced
balance factor between +1,0,-1
balance factor=height of left subtree-height of right subtree
keep balance=high performance

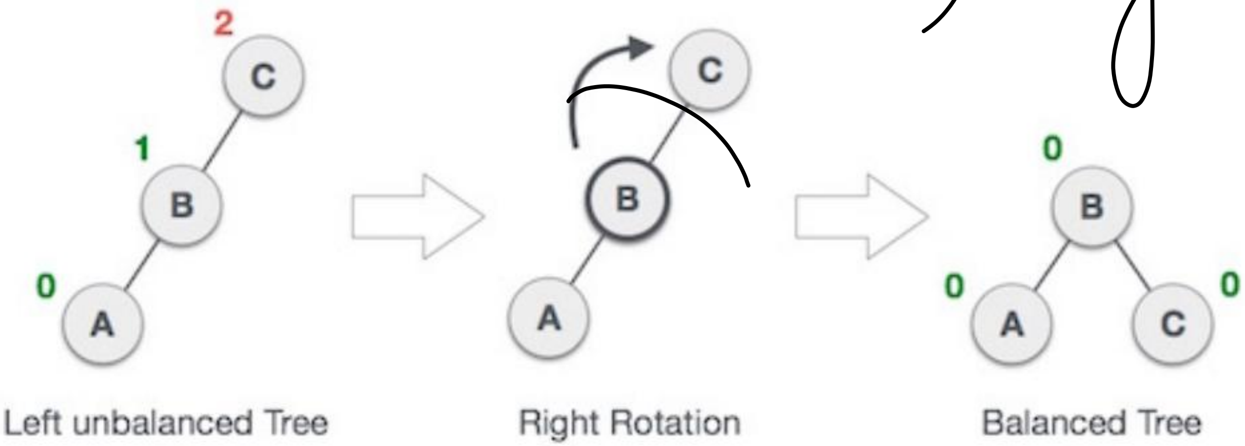
to construct: add one node at a time in bst.
->1 node at a time inserted and calculate balance factor from bottom to top
->stop on first location where unbalance(b.f. goes beyond +1 to -1)
->make path from unbalanced to newly inserted that caused unbalance
and see which case it is for rotation.(path only of 3 nodes)

Rotations

right of right

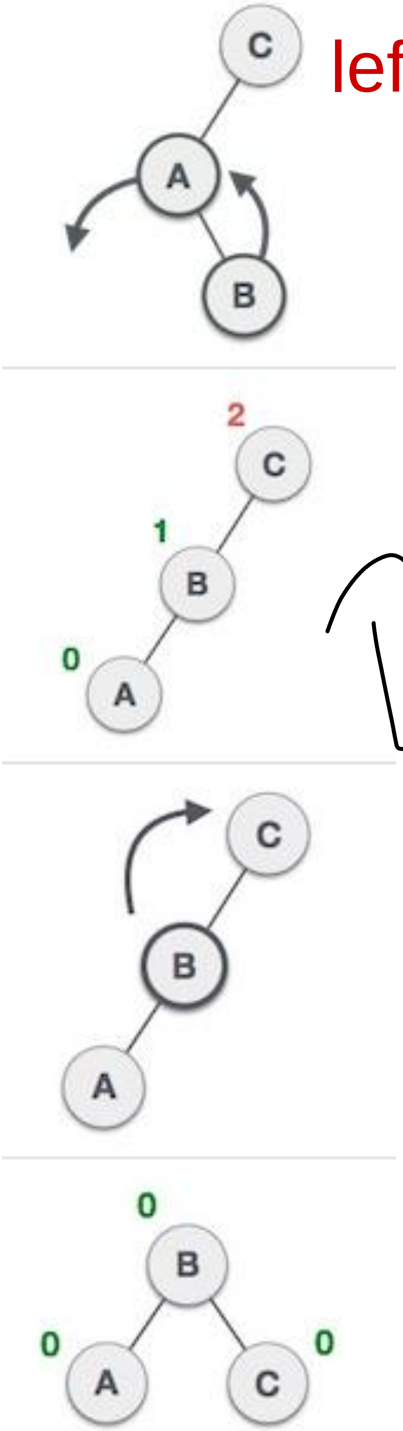


left of left

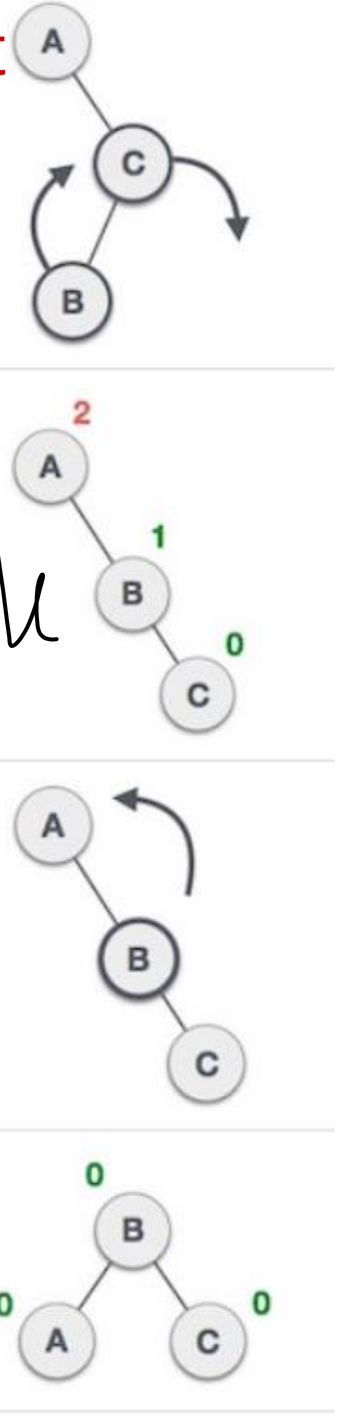


Single

right of left



left of right



Double

B-Tree

- B-Tree is a self-balancing search tree.
- Single node consist multiple elements this reduces number of read write and memory swaps.
- the B-Tree node size is kept equal to the disk block size
- All leaves are at the same level.
- B-Tree is defined by the term minimum degree ‘M‘.
- M is odd. M = 5
- On Order M:
    - Maximum M-1 elements at a node 4
    - Minimum M/2 elements at a node 2
    - Split when M elements reached 5
- All keys of a node are sorted in increasing order.
- B-Tree grows and shrinks from the root which is unlike Binary Search Tree.
- Like other balanced Binary Search Trees, the time complexity to search, insert and delete is O(log n).
- Insertion of a Node in B-Tree happens only at Leaf Node in order and BST form.

B+ Tree

BST IMPLEMENTATION

- Static Array
- Dynamic Linked List
- Dynamic Tree node